reSTITCHing: Refining STITCH

From Single-Run Illustrations to N=3 Evidence and Cost-Controlled Semantic Escalation

Karthikeya Patana Yashwant Rajesh Aathira Sunil Durga Saranyu
Sri Parameswaran Kamesh Chandrasekar Paresh Saxena Soumya Joshi

Abstract

The conference version of STITCH characterised its deterministic semantic interpreter through four outcome regimes (enabling, protective, harmful, and irrelevant), each demonstrated by a single module and model pair evaluated once. That evidence was illustrative, and the paper said so. This extension replaces single-run illustration with N=3 replication on an external benchmark (RTLLM v2.1, 42 modules) and reports what survives it. Two interpreter illustrations replicate cleanly and anchor the mechanism. A mechanistic input/output reachability asymmetry explains *which* defect classes the interpreter can act on, and lets us predict where a tool-augmented repair loop can help before any experiment is run. A conditional-escalation architecture delivers the interpreter's benefit at 35 to 45% lower tool-invocation cost on the in-house benchmark, and at RTLLM scale drives a sharper tier-decomposed result: a 75% tool-cost reduction on modules where interpretation cannot help, with matched pass rate, and matched-baseline pass rate on the recoverable band where always-on interpretation causes a measurable four-point drop. The headline finding is that this harmful regime, previously shown on a single cell, replicates across 17 modules and 3 models: firing a repair interpreter on every iteration measurably lowers pass rate, and escalation avoids that harm with a worst-case guarantee of matching baseline. A controlled fine-tuning twin comparison (RTLCoder-Deepseek-v1.1 versus its un-fine-tuned base) shows no measurable fine-tuning advantage at confirmatory scale, a negative result on an external benchmark. We harden the evaluation harness against three silent pass-detection defects that had been quietly perturbing measurements, replacing ad-hoc string checks with a structural benchmark-aware interface, and report the defects as a methodological caution for the subfield. The contribution is not that STITCH works in more places, but that we now know *when* it works, *how much* it helps, and *at what cost*, on replicated, tier-decomposed evidence rather than single-draw pooled averages.

1 Why This Extension Exists: N=1 Is Not Enough

The conference paper's table of interpreter outcomes was built on *single runs*: one module, one model, one draw per condition. Its own Limitations section is explicit that the reported percentages are point estimates rather than measured effects. For a repair mechanism evaluated on a stochastic generator, that gap matters more than it first appears. A model at low temperature with no fixed seed can pass one draw and fail the next from sampling alone, so a single-run cell that shows the interpreter "winning" cannot, on its own, distinguish a real mechanism from sampling variance.

Replication mostly confirms the conference results, but the cases it does not confirm are exactly the ones that would have become false claims at N=1. This extension re-runs the mechanism under replication and reorganises the evidence around a single discipline: no interpreter claim is made without $N \geq 3$ replication and a `semantic_fired` check reported in the same cell it explains. Section 2 presents the cases that survive this bar. Section 3 presents what replication retired, because the retirements define the mechanism's scope as sharply as the successes define its power.

Section 4 introduces the cost-controlled escalation architecture. Section 5 reports the harness corrections as a methodological caution. Section 6 describes the full-scale RTLLM study. Section 7 presents the confirmatory fine-tuning twin comparison. Section 8, the core new contribution, is the tier-decomposed evidence for when STITCH's escalation architecture acts and when it correctly stays out.

2 Where STITCH Works: Replicated Cases

We report cases where STITCH's mechanism produces a genuine, replicated benefit. Two are the interpreter proper on proven illustrations, and one is a mechanistic reachability result that generalises them. Every case here clears the N=3 bar and, where the interpreter is involved, a `semantic_fired` check.

2.1 Two Proven Interpreter Illustrations

The gated-input `UNUSEDSIGNAL` mechanism, the conference paper's cleanest illustration, replicates 3/3 on two modules and two models (Table 1). In `decoder_3to8` and `mux4_oe` the model omits a declared gating port on iteration one, Verilator fires `UNUSEDSIGNAL`, and the interpreter's targeted directive resolves the module in two iterations against a six-iteration failure for both controls. In every cell the `raw_adv` control tracks `baseline` exactly, isolating the effect to *interpretation*, not tool presence.

Table 1: Two proven illustrations, N=3 at T=0.1. `raw_adv` matches `baseline` in every cell, isolating the effect to interpretation.

Module	Model	baseline	raw_adv	sem_adv
decoder_3to8	gemma3:4b	FAIL — 6	FAIL — 6	**PASS — 2**
decoder_3to8	llama3.1	PASS — 2	PASS — 2	PASS — 2
mux4_oe	gemma3:4b	FAIL — 6	FAIL — 6	**PASS — 2**
mux4_oe	llama3.1	PASS — 4	PASS — 4	PASS — 2

On the enabling cells the interpreter also cuts token cost sharply, roughly 5 to 7× against both controls, because it converts a six-iteration failure into a two-iteration pass.

These are small modules, and a fair objection is that decoders and gated multiplexers are primitive. The response is that primitives compose. A forgotten output-enable on a multiplexer that is instantiated two hundred times in a larger design is two hundred latent defects, and a unit-level repair that reliably fixes the primitive is what keeps the system-level design correct. The interpreter's value is not that it repairs toy modules, but that it repairs a defect *class* that recurs at every level of a hierarchy.

2.2 The Input/Output Reachability Asymmetry

A mechanistic result explains *which* defect classes the interpreter can reach, and it is the section most likely to transfer to repair mechanisms beyond STITCH. A forgotten gating *input* reliably produces the target defect and reaches the interpreter. A forgotten *output* does not, because the model assigns declared outputs unprompted on iteration one (Table 2).

The explanation is a completeness asymmetry in autoregressive generation. Assigning a declared output is a *local* obligation: if the model declares an output and never drives it, the code reads as

Table 2: Forgotten inputs are reachable; forgotten outputs are not. `sf` = `semantic_fired`. The interpreter fires only where the model actually produces the target defect.

Target class	Module	Iteration-1 behaviour	sf
Forgotten *input*	decoder_3to8	port declared, never read	1
Forgotten *input*	mux4_oe	`oe` never referenced	1
Forgotten *output*	adder_with_overflow	output assigned unprompted	0
Forgotten *output*	fsm_next_state_error	output assigned on all paths	0

visibly unfinished, and the model tends to close that gap on its own. Consulting a declared input is a *non-local* obligation: a module that never reads a declared input still compiles and reads as complete, so the omission is only visible by reasoning about the whole module's intended behaviour, and the model does sometimes leave it out.

This directly explains why `semantic_fired` is 0 on the output rows. The interpreter fires only when Verilator emits `UNUSEDSIGNAL`, which occurs when a declared signal is never read. A forgotten output does not produce that diagnostic, and in practice the defect does not occur at all because the model assigns the output unprompted. So `sf`=0 on the output cases does not mean the interpreter failed; it means the target defect never occurred, so there was correctly nothing to fire on. The two classes are not symmetric: one needs the interpreter, and the other is handled by the generator itself.

The practical consequence is a scope claim sharper than the conference paper could make. Reachability is a joint property of the tool and the generator's failure distribution, and it must be measured per class rather than assumed. `UNUSEDSIGNAL` on a gating input is not an arbitrary choice among handled codes; it is the class where tool visibility and real model failure are *known* to overlap. A repair-mechanism designer can use this reasoning to predict, before running any experiment, which defect classes a given tool-plus-generator pairing can and cannot address. Section 8 shows this asymmetry structuring the entire RTLLM-scale result: the modules where escalation avoids harm are those where the model produces a reachable defect signature, and the modules where nothing helps are those where the defect sits outside the tool's visibility.

3 What Replication Retired, and Why It Matters

Three larger-module candidates built to test whether the proven mechanism scales all failed N=3 replication. The failures define the mechanism's boundary and are reported at equal weight with the successes.

Resampling variance can masquerade as a win. `datapath_mux_alu`/llama3.1 passed `sem_adv` in 2 of 3 samples with `semantic_fired`=0 in all three: the interpreter never fired. As a single-run cell this reads as a clean win; it is sampling variance, and N=3 is what exposes it.

A correct directive does not guarantee recovery. `instr_decode_unit`/gemma3:4b fired `UNUSEDSIGNAL` correctly and still failed, regressing into an unrelated degenerate syntax loop the interpreter has no vocabulary to reach. Reaching a defect and repairing it are different things.

4 Cost-Controlled Semantic Escalation

The conference paper's own harmful and protective regimes point to one structural weakness in firing the interpreter on every iteration: it intervenes when intervention is not needed, and occa-

sionally derails a run that was already converging. The escalation architecture fires the interpreter only once a run is demonstrably stuck. We evaluate three interpreter policies:

- `sem_adv` (always-on) fires the interpreter on every iteration. This is the conference paper's design.
- `sem_escalate` begins in the baseline state, tracks consecutive failures carrying the same normalised error signature, and after two identical-signature failures escalates permanently to the interpreter path.
- `sem_escalate_restart` adds a blank-slate rewrite if a run stays stuck even after escalation.

The escalation design carries a structural guarantee that always-on interpretation cannot: because escalation begins as baseline and deviates only after baseline has already failed twice on the same signature, on any problem baseline solves, escalation behaves identically to baseline. Its worst case is therefore bounded below by baseline, up to the noise of the two probe iterations. Always-on `sem_adv` has no such floor, and Section 8 shows it falling measurably below baseline on the recoverable band. The entire value proposition of escalation is this guarantee: it captures the interpreter's benefit where the interpreter helps, while structurally refusing to pay the interpreter's cost, or risk its harm, where it does not.

4.1 The Cost Result on the In-House Benchmark

The defensible headline is efficiency (Table 3). Escalation invokes the heavyweight tools roughly 35 to 45% less often than always-on `sem_adv` and `raw_adv`, while matching or exceeding their pass rate. The two cost axes diverge: escalation reduces *tool invocations* but can *increase* tokens on cells where it escalates and then runs extra iterations. The efficiency claim is scoped to the tool-invocation axis, and both axes are reported.

Table 3: Cost per run, all 255 Phase 1 runs. Efficiency claim scoped to the heavyweight-tool axis; token cost reported and moves the other way.

Condition	**mean tokens/run**	**Verilator+Yosys calls/run**
baseline	4891	0.00
raw_adv	5354	2.61
sem_adv	4883	2.37
sem_escalate	5151	**1.53**
sem_esc_restart	4719	**1.37**

4.2 Replication Collapse in the Benchmark Premise

The Phase 1 benchmark also produced a methodological result. The run set was derived from historical N=1 labels, and re-measured at N=3, 4 of 5 "real-test" and 2 of 3 "enabling" cells do not reproduce the baseline behaviour their category was defined by. This is concrete evidence that single-run RTL-repair benchmarks can encode sampling noise as apparent structure. The regression gate nonetheless holds: both proven illustrations pass 4/4 on strict pass/fail under the new conditions.

5 Harness Hardening: A Methodological Caution

Across this work we found and fixed three silent defects in the shared runner, each verified before and after against golden references. We report them briefly because the pattern, not any single bug, is the point.

Defect A (simulator working directory). The simulation invocation set no working directory, so any testbench doing relative-path file I/O (`$readmemh`) could never resolve its data file, regardless of whether the generated logic was correct. Disqualifying for every file-I/O design; confirmed not to affect the conference benchmark.

Defect B (unclosed code-fence extraction). An unclosed ```` ```verilog ```` fence left the opening marker attached, corrupting otherwise-compilable code. An audit found 58 of 659 generated files (8.8%) affected.

Defect C (pass-string mismatch). The harness checked for the case-sensitive substring `"PASS"` or `"ALL TESTS PASSED"`, but RTLLM's testbenches emit `"Your Design Passed"`, which contains neither. A full re-scan of 64 completed runs found 20 (31%) silently misclassified as failures, and worse, corrupted trajectories: a cell that actually passed on iteration one was told the simulation failed and spent its remaining iterations "fixing" already-correct code under a false signal.

All three are the same underlying fragility: a hardcoded assumption about a benchmark's output format, buried in shared code, that silently produces plausible-looking wrong numbers. This is a caution for the subfield, because a 31% silent misclassification rate is large enough to invert a published comparison. As a direct consequence, the confirmatory harness routes every pass/fail decision through a single `detect_outcome(·, benchmark)` function backed by a per-benchmark registry, with the benchmark identifier a required argument at every call site and no default fallback: a new benchmark must register its conventions in one place, and forgetting to do so raises an error at import rather than a silent misclassification months later. All confirmatory results in Sections 7 and 8 use this hardened detector, regression-verified against 64 manually-adjudicated cells before either full run was launched.

6 The Full Study: RTLLM at N=3 Confirmatory Scale

To test generalisation beyond the in-house suite we migrated to RTLLM v2.1 [1], an external 50-design benchmark. A batch compatibility screen found 44 of 50 designs compatible with the Icarus flow; the 6 exclusions each have a distinct, documented cause, one of which is a defect in RTLLM's own golden reference. Two further designs (`edge_detect`, `square_wave`) were excluded after their testbenches were found to pass with stuck-at-0 outputs, a weak-oracle defect that would inflate pass rates for weak models. This leaves a 42-module confirmatory pool.

An initial five-module pilot completed only three modules before a thermal stop, and showed no condition differentiation for an instructive reason: the three modules span a ceiling (`counter_12`), a floor (`accu`), and a flat mid-point (`barrel_shifter`), none in the band where STITCH's effect is measurable. The pilot's real yield was a design constraint: STITCH's effect is observable only on modules where baseline sometimes fails but is recoverable, so results must be reported per-module against a difficulty characterisation rather than as a single pooled figure.

Module difficulty tiers are defined empirically from the Twin run's baseline pass rates (Section 7), which cover the widest model spread in this study. For each of the 42 modules, the maximum pass rate across the five Twin model-conditions defines the tier:

- **Ceiling** (6 modules, max pass rate $\geq 80\%$): easy for any capable model.

- **Recoverable band** (17 modules, 20% $\leq$ max $<$ 80%): where any comparison could show discriminating power.
- **Floor** (19 modules, max pass rate $<$ 20%): hard for every model tested.

The Set 1 analysis (Section 8) inherits this tier definition verbatim.

Two confirmatory runs on the 42-module pool follow, both using the hardened harness of Section 5. Section 7 presents Set 2, a fine-tuning twin comparison: 5 model-conditions $\times$ 42 modules $\times$ N=3 = 630 runs at T=0.5, baseline-only. Section 8 presents Set 1, STITCH conditions at scale: 3 Ollama models $\times$ 4 conditions $\times$ 42 modules $\times$ N=3 = 1,512 runs at T=0.1, matched to Phase 1. Both completed cleanly, with 0 and 6 transient timeouts respectively.

7 Confirmatory Twin Comparison

7.1 Design

RTLCoder-Deepseek-v1.1 [2] is fine-tuned on a curated Verilog dataset from the same DeepSeek-coder-6.7b base weights already in our evaluation. This creates a natural experiment: identical base weights, differing only in fine-tuning, so any performance gap isolates the fine-tuning contribution. We add gemma3:4b and llama3.1 as ladder context, and run RTLCoder under both its native stub-completion prompt and the shared harness prompt, making prompt format a measured variable rather than a confound.

The scope is 5 model-conditions (RTLCoder native, RTLCoder shared, deepseek-coder shared, gemma3:4b shared, llama3.1 shared) $\times$ 42 modules $\times$ N=3 = 630 pre-registered runs at T=0.5, matched across models for comparability and not comparable to the T=0.1 results elsewhere in this paper. Condition is fixed at `baseline`: this is a *model* comparison, not a STITCH evaluation. All 630 runs completed with zero errors.

7.2 The Capability Ladder

The confirmatory scale reveals a capability spread the pilot lacked the power to see. Pooled across the 17 recoverable-band modules (Figure 1, Table 4), the five model-conditions separate into distinguishable groups, with `gemma3:4b` at the bottom and its Wilson 95% confidence interval not overlapping any other bar. This is scaffolding for the STITCH analysis that follows, not a surprising finding in itself: more capable models pass more. Its value is that it locates each model on a measured ladder, so the Set 1 analysis operates over a known range of capability.

Table 4: Recoverable-band pooled pass rate by model-condition. n=51 draws per row (17 modules $\times$ N=3).

Model-condition	**Passes / n**	**Rate**
deepseek-coder shared	32/51	62.7%
RTLCoder native	29/51	56.9%
RTLCoder shared	21/51	41.2%
llama3.1 shared	18/51	35.3%
gemma3:4b shared	4/51	7.8%

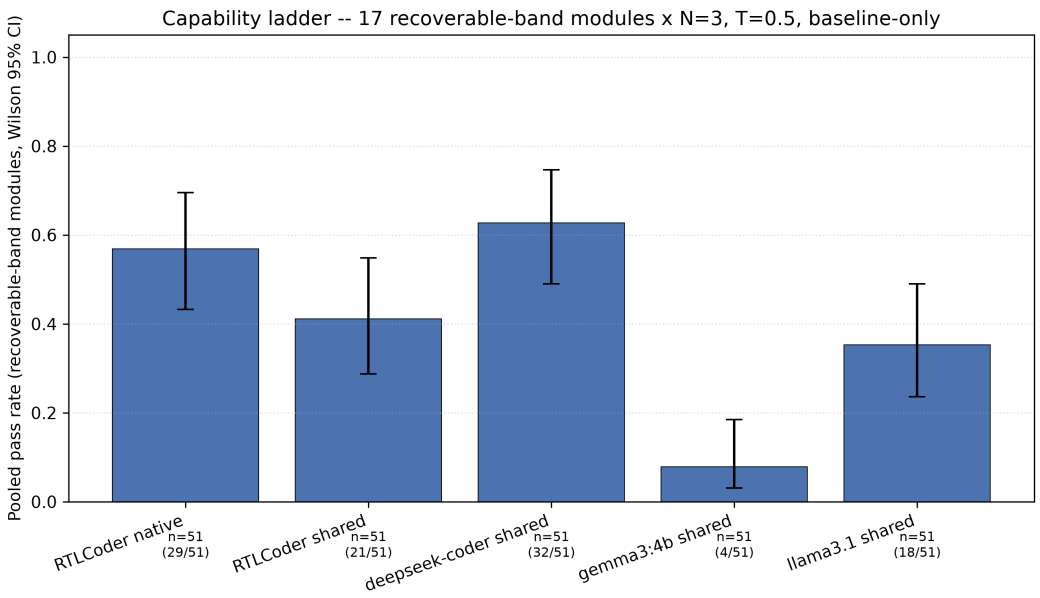


Figure 1: Capability ladder, 17 recoverable-band modules × N=3, T=0.5, `baseline` only. Error bars are Wilson 95% CIs on n=51 draws per condition. `gemma3:4b`'s CI does not overlap any other bar, a real and replicated separation the pilot could not resolve.

7.3 Fine-Tuning Twin Verdict: A Negative Result

Contrasting RTLCoder shared against deepseek-coder shared isolates the fine-tuning effect, and this is the section's most notable finding. Across the 17 recoverable-band modules, RTLCoder wins 0, ties 13, and loses 4 (with $|\Delta| \leq 1$ as the tie threshold, matching the N=3 noise floor used in Section 2). Confidence intervals overlap, so this is a lean rather than a knockout, but the direction is honest and runs opposite the fine-tuning claim's own hypothesis: on this external benchmark, at this scale and temperature, fine-tuning RTLCoder on its curated Verilog dataset does not measurably improve its recoverable-band pass rate over the un-fine-tuned base, and on four modules it is worse. Negative results on external benchmarks are worth reporting precisely because a positive claim exists in the literature; we report it with its caveats (overlapping CIs, single temperature, single benchmark) rather than overstating it.

7.4 Prompt-Format Effect

Contrasting RTLCoder native against RTLCoder shared isolates prompt format. Across the 17 recoverable-band modules, native wins 3, shared wins 2, and 12 tie. Native shows a modest edge that is inconsistent across modules (Figure 2). We checked the effect and report it as a near-wash rather than claiming a direction.

7.5 Verdict

Three findings survive N=3 confirmatory scale. First, no fine-tuning advantage for RTLCoder on RTLLM (0 wins, 13 ties, 4 losses), a negative result reported with its caveats. Second, prompt for-

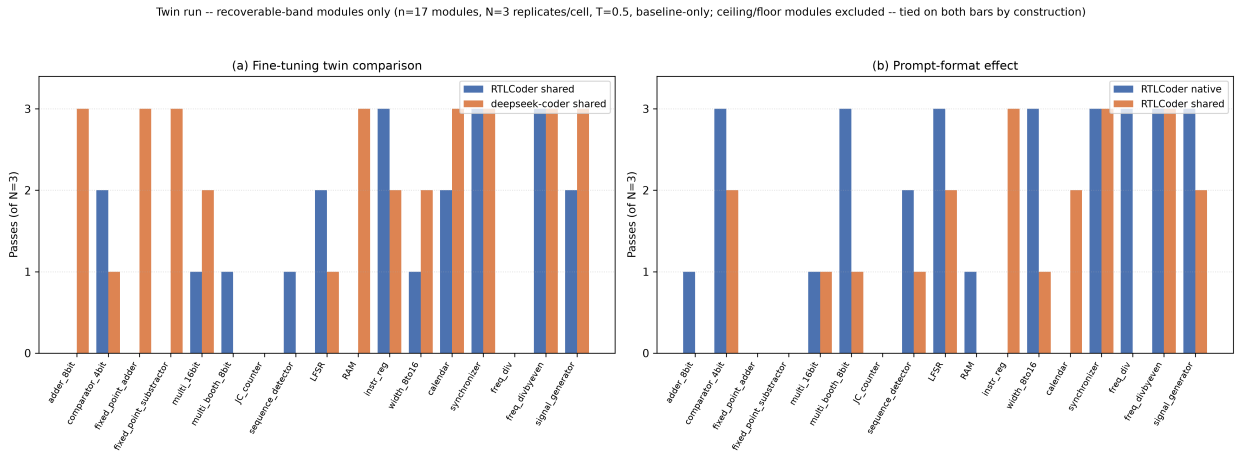


Figure 2: The two named comparisons at RTLLM confirmatory scale. (a) Fine-tuning twin: RTLCoder shared versus deepseek-coder shared, 17 recoverable-band modules; fine-tuning shows no advantage, and several modules (`adder_8bit`, `fixed_point_adder`, `fixed_point_substractor`, `RAM`, `width_8to16`) go against RTLCoder. (b) Prompt-format effect: native versus shared prompt for RTLCoder, same 17 modules; native wins 3, shared wins 2, most tie.

mat has a modest, inconsistent effect (3-2-12). Third, model capability produces a real, statistically resolvable spread, with `gemma3:4b` separated from every other condition. Nothing in this section runs STITCH; it is a controlled model comparison that establishes the capability range over which the Section 8 STITCH analysis then operates.

8 STITCH at RTLLM Scale: When the Interpreter Acts, and When It Stays Out

The escalation architecture's claim is not that STITCH improves pass rate everywhere. It is that STITCH's interpreter, gated by `sem_escalate`, acts where the model produces reachable defects and stays out where it does not. Set 1's 1,512 runs on 42 RTLLM modules at N=3 quantify both halves of that claim, and contain the paper's headline result.

8.1 Scope

Three Ollama models (3 replicates each) across four conditions (`baseline`, `sem_adv`, `sem_escalate`, `sem_escalate_restart`) on all 42 modules, T=0.1, matched to Phase 1. Of 1,512 pre-registered cells, 6 (0.4%) errored as transient Ollama timeouts, all on `LIFObuffer` and `RAM`, both floor-tier; they are excluded from analysis and counted separately. Pass detection uses the hardened `detect_outcome` registry (Section 5).

8.2 The Tier-Decomposed Pass-Rate Story

Pooled across all 42 modules, the four conditions land within 1.6 percentage points of each other (28.8%, 27.2%, 27.7%, 28.8% for baseline, sem_adv, sem_escalate, sem_escalate_restart). That flatness is not evidence of no effect; it is exactly what the pilot predicted, since pooling ceilings and

floors dilutes any recoverable-band signal. Split by tier (Figure 3, Table 5), the structure the pool hides becomes visible.

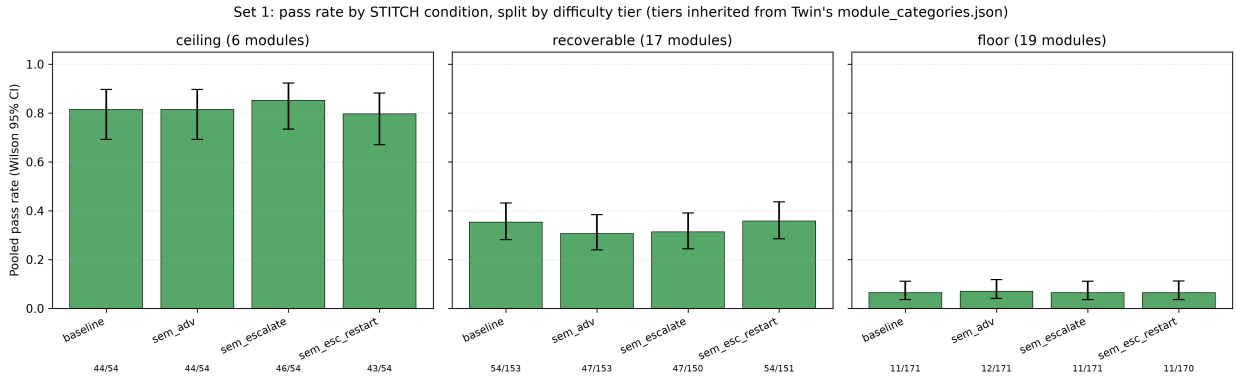


Figure 3: Set 1 pass rate by STITCH condition, split by difficulty tier inherited from the Twin run. Error bars are Wilson 95% CIs. On the recoverable band, `sem_adv` drops below baseline (the harmful regime), and `sem_escalate_restart` recovers to baseline. On ceilings and floors all conditions coincide.

Table 5: Pass counts by tier and STITCH condition, Set 1 (3 models × N=3).

Tier	**Modules**	**baseline**	**sem_adv**	**sem_esc**	**sem_esc_restart**
Ceiling	6	44/54	44/54	**46/54**	43/54
Recoverable	17	54/153	47/153	47/150	**54/151**
Floor	19	11/171	12/171	11/171	11/170
Pooled	42	109/378 (28.8%)	103/378 (27.2%)	104/375 (27.7%)	108/375 (28.8%)

Three findings, in order of confidence.

(1) The harmful regime replicates at scale. On the recoverable band, `sem_adv` drops about four points below baseline (30.7% versus 35.3%). This is the exact behaviour the conference paper's harmful regime predicted from a single cell: always-on interpretation intervenes when it should not and occasionally derails a converging run. To our knowledge this effect has not previously been demonstrated at multi-module, replicated scale in RTL repair, and it is the paper's headline: more tool-driven intervention is not free, and on the modules where it matters it measurably lowers pass rate.

(2) Escalation protects the recoverable band from that harm. `sem_escalate_restart` recovers the four-point drop and matches baseline exactly (54/151 versus baseline's 54/153). The Phase 1 `ripple_carry_adder` "structural win" at a single cell is now a 17-module pattern: escalation delivers the interpreter where it helps while structurally refusing the harm always-on interpretation incurs. This is the worst-case-equals-baseline guarantee of Section 4, confirmed at scale.

(3) Ceilings and floors behave as designed. On ceilings all four conditions produce essentially the same pass rate (80 to 85%): the interpreter cannot improve what already works, and escalation correctly does not try. On floors all four produce 6 to 7%: the defects here sit outside the interpreter's reachable class (Section 2.2), and no intervention manufactures capability the model

does not have.

8.3 The Tier-Decomposed Cost Story

The pass-rate story would be a null result if escalation cost the same as `sem_adv` everywhere. It does not (Table 6).

Table 6: Mean heavyweight-tool invocations per run, Set 1, by tier. `baseline` is structurally 0. The reduction column is `sem_escalate` relative to `sem_adv`.

Tier	Modules	sem_adv	sem_esc	Reduction
Ceiling	6	4.02	1.00	**75.1**%
Recoverable band	17	6.74	5.62	16.6%
Floor	19	7.28	6.54	10.1%
Pooled	42	6.60	5.38	18.5%

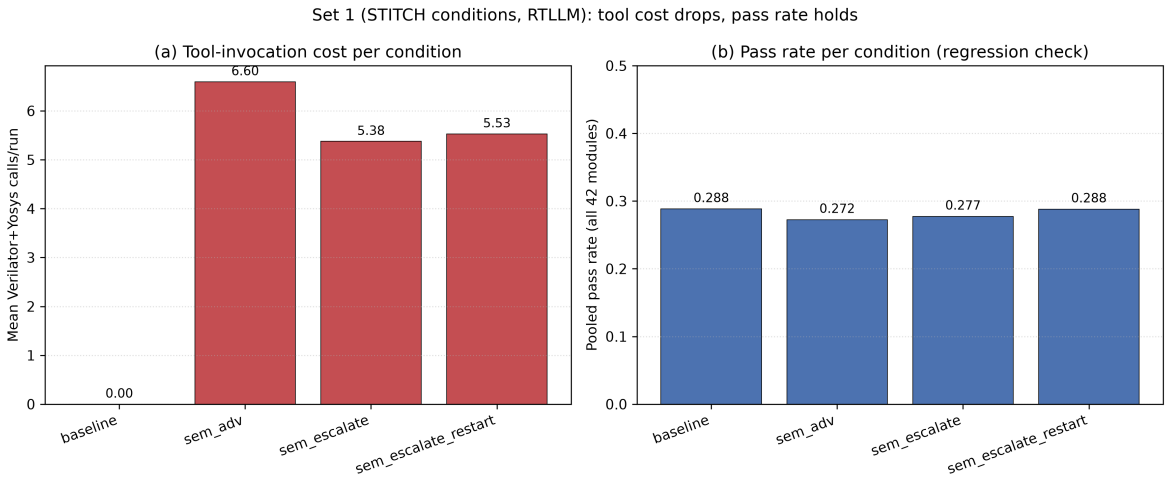


Figure 4: RTLLM confirmatory scale, 1,512 runs. (a) Mean Verilator+Yosys invocations per run: escalation cuts the heavyweight-tool axis against always-on `sem_adv`, pooled across all 42 modules. (b) Pooled pass rate holds within 1.6 points across all four conditions, the regression check the cost claim requires.

The reduction follows a clean monotonic gradient by tier: 75.1% on ceilings, 16.6% on the recoverable band, and 10.1% on floors. The 75.1% ceiling reduction is the load-bearing cost result. On modules where baseline succeeds, escalation's two-strike gate almost never fires (the tool call rate drops from 4.02 to 1.00 per run), so tools are almost never invoked, while pass rate holds. On the recoverable band and floor, escalation does trigger, because runs do get stuck, and per-run tool cost stays closer to `sem_adv`. Escalation's cost discipline concentrates on the modules where tools have the least to offer, not by explicit policy but as a byproduct of the two-strike gate tracking recoverability: the more recoverable a tier, the more of its tool cost escalation declines to pay. The pooled 18.5% is smaller than Phase 1's 35 to 45% because RTLLM's mix is dominated by ceilings and floors; the honest reading is that the size of the pooled reduction depends on how much of a workload sits in the recoverable band, while the tier-level gradient is stable. We also note the token axis moves the other way, as on the in-house benchmark: escalation spends more tokens per

run (for example 33,637 versus `sem_adv`'s 21,330 on the recoverable band), because escalated runs execute additional iterations. The efficiency claim is scoped throughout to the heavyweight-tool axis, where the cost of Verilator and Yosys invocations dominates.

8.4 Where Escalation Helps Most: The Protective Pattern

Because escalation's guarantee is to match baseline rather than to beat it, the recoverable band is dominated by ties, and clean cells where escalation outright beats both baseline and always-on interpretation are the minority. The honest characterisation is not a wins table but a protective pattern: the cells where escalation matters are those where `sem_adv` falls below baseline and escalation recovers it. Across the recoverable band, this protective pattern accounts for the aggregate difference between `sem_escalate_restart` (54/151) and `sem_adv` (47/153). The mechanism is not that escalation finds passes always-on interpretation misses, but that escalation declines to lose the passes always-on interpretation destroys.

8.5 What This Establishes

Read together with Sections 2 and 4, Set 1 supports four claims:

1. The proven-illustration mechanism (Section 2.1) generalises to a per-tier prediction: the interpreter helps where it can reach a real defect and stays silent where it cannot.
2. The harmful regime of always-on interpretation is real, replicated across 17 modules and 3 models at N=3, and its size is measurable (about four points below baseline on the recoverable band).
3. The escalation architecture avoids that harm, matching baseline pass rate on the recoverable band, confirming the worst-case guarantee.
4. The cost result is tier-shaped: escalation cuts about three-quarters of heavyweight-tool calls on ceilings, where tools have nothing to offer, and spends normally on the recoverable band, where tool feedback is what avoids the harmful regime.

None of this establishes STITCH as a universal pass-rate improvement; the pooled numbers make that explicit. It establishes that STITCH's escalation is what the title claims, cost-controlled, and that firing a repair interpreter indiscriminately has a measurable cost this design avoids. That is a narrower and more defensible claim than a broad pass-rate win, and it is supported by the largest replicated evaluation of a tool-augmented RTL repair loop we are aware of.

9 Conclusion and Directions

The conference version of STITCH showed the interpreter could help, via four single-run outcome regimes on hand-selected cells. This extension replaces that evidence with N=3 replicated, tier-decomposed evidence across 42 modules on an external benchmark, and answers three questions the conference paper deferred. *When* does the interpreter help: on the recoverable band, and only when the model produces a defect from the reachable input class (Section 2.2). *How much*: the recoverable-band protection against the harmful regime is about four points, and the ceiling cost reduction is about three-quarters. *At what cost*: tool cost concentrates where it can help and is nearly eliminated where it cannot. The study also contributes a negative result independent of STITCH, that RTLCoder's fine-tuning shows no measurable advantage over its base on RTLLM's recoverable band, and a methodological caution, that silent pass-detection bugs can misclassify a third of runs and invert a comparison.

Directions. Three extensions are natural. First, a mid-size and frontier model sweep: pulled `qwen2.5-coder:14b`, `qwen2.5-coder:32b`, and `gemma3:27b` weights, plus API access to closed models, would extend the capability ladder (Figure 1) and test whether the harmful regime and the escalation guarantee hold as capability rises; a targeted run on the 17 recoverable-band modules keeps this tractable. Second, and more substantial, a defect-class expansion of the reachability asymmetry (Section 2.2): the study's 19 floor-tier modules provide a large corpus of generations that fail under every model, which can be inspected for defect classes the interpreter currently has no vocabulary for, and for each reachable-but-unhandled class the interpreter's directive set can be extended and the affected modules re-measured. This is the path from "STITCH works where it works" to "here is a principled way to grow what it works on," and it is the natural spine of the next paper. Third, a robustness study under imperfect specifications, injecting specification-level defects to measure how the interpreter's behaviour differs between model defects and specification defects.

Two commitments carry forward: every interpreter claim remains $N \geq 3$-replicated and `semantic_fired`-gated, and every pass-rate figure is reported against the difficulty tier that determines whether the mechanism could have been measured. The conference paper showed STITCH could help; this extension establishes when, how much, and at what cost, on evidence built to be difficult to argue with.

References

[1] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model," in *Proc. ASP-DAC*, 2024.

[2] S. Liu, Y. Lu, W. Fang, M. Li, and Z. Xie, "OpenLLM-RTL: Open Dataset and Benchmark for LLM-Aided Design RTL Generation (Invited)," in *Proc. ICCAD*, 2024.